

When Does KV-Cache Eviction Pay on a Phone?

μ KV: sequence-level, attention-scored eviction that never leaves the FlashAttention path

Md Romyull Islam

Kennesaw State University
Marietta, Georgia, USA
romyullislam2012@gmail.com

ABSTRACT

KV-cache eviction is designed on server GPUs and justified by one number: how much cache it drops. We port nine policies into one mobile engine and measure them end-to-end on a OnePlus 15 — CPU and GPU, each at its *own published configuration* — and find three assumptions that break on the device. *Budgets do not materialize*, a *realizability gap*: on the single shared cell array on-device engines use, a per-head evictor frees a cell only if every head dropped it, so SnapKV, asked for 1024 cells, retains 89–100%, and 6 of 15 of its generations are byte-identical to the full cache’s. *The standard implementation route corrupts*: reading attention mid-graph forces a scheduler split that silently corrupts output on Adreno/Vulkan (isolated with a three-way control). *And whether eviction pays at all is set by the KV/W ceiling, the model-backend pair rather than the policy*. The same configuration is $4.80 \pm 0.07\times$ vanilla on the phone’s CPU ($n=3$) and $1.20\times$ on its GPU, where a weight-bandwidth roofline caps *any* evictor at $1.40\times$; on Phi-3 it is $2.64\times$ against a $2.83\times$ ceiling. To our knowledge, the first end-to-end account (throughput, energy, temperature, cells *actually* freed) of published eviction on a phone. μ KV is the design these force: scores from a side node *inside* the FlashAttention graph (prefill +14% cold, at vanilla parity sustained and on the GPU, where FA-off prefill costs +33–111%), one keep-set shared by all heads so eviction frees real cells, compacted in place. It retrieves 14/14 needles at 13.0% retention where StreamingLLM gets 7/14, and matches KeyDiff’s quality at its 6K budget on $3.3\times$ fewer live cells. We report negatives as plainly as wins, including our own corrected measurement errors.

KEYWORDS

mobile LLM inference, KV cache, eviction, FlashAttention, energy

1 INTRODUCTION

Long-context LLMs on phones are gated by the KV cache: decode reads it every step, and on a 12–16 GB device it

competes with every foreground app. A rich eviction literature exists (H2O [1], TOVA [2], SnapKV [3], Ada-KV [4], StreamingLLM [5]), all designed and scored on server GPUs. The systems that *do* run on phones move bytes rather than discard them: KVSwap to disk [8], DynaKV to UFS [9], MobiLoRA across adapters [10]; none evicts, none reads the battery.

This paper asks the question the literature skips: **what actually happens when published eviction policies meet a real phone?** We mean it in end-to-end throughput, energy, temperature, and memory *actually* retained. Prior work stops short: eviction papers evaluate quality on server GPUs (KeyDiff [6] reports only scoring-kernel latency on a phone), and mobile characterization studies measure energy and thermals without varying the eviction policy. We port nine policies into llama.cpp [11], run each at its *own published configuration* on a OnePlus 15 (Snapdragon 8 Elite Gen 5: CPU, Adreno 840), and measure every axis a deployment decision needs (throughput by phase, perplexity, retrieval, LongBench F1, retained cells, RSS, system energy, temperature) under a cooling and pinned-scheduler discipline we found necessary along the way. Four findings organize the paper:

- (1) **The realizability gap** (§2): published budgets do not materialize on-device. On-device engines keep one cell array shared across heads and layers; a cell is freed only when *every* per-head selector drops it. SnapKV’s budget retains 91.3% of cells on Llama-1B (CPU, FA-on), 94.5% on Phi-3; H2O’s 20%-of-context ratio retains 94–100%, so all of them pay eviction’s costs and collect none of its benefit.
- (2) **The graph split** (§2): the standard implementation route is numerically unsafe on the phone GPU. Intercepting the softmax node forces a scheduler graph split that corrupts the computation on Adreno/Vulkan, giving 10–12.5% degenerate tokens even when nothing is evicted. Scores must be a *graph output*.
- (3) **The KV/W ceiling** (§4): the model-backend pair decides what eviction is worth. Decode is bandwidth-bound on the GPU: with weights W read every step,

One shared cell array, three eviction classes: only a sequence-level, attention-scored keep-set both compacts and keeps the needle

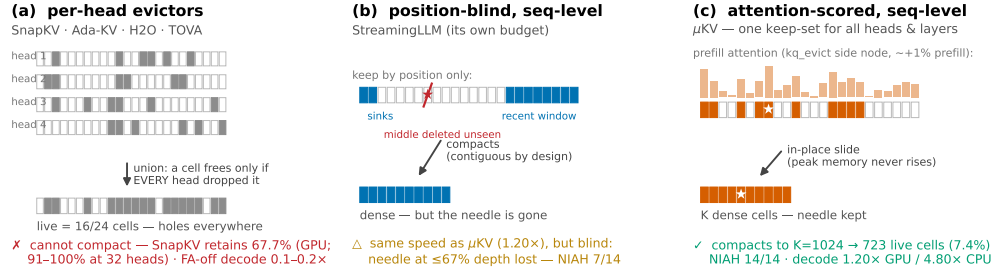


Figure 1: One shared cell array, three eviction classes: (a) per-head union, (b) position-blind, (c) μKV. Verdict lines are measured.

the best any policy can do is $(W+KV)/(W+KV/r)$. At Llama-1B/16K that caps *any* evictor at 1.40× (measured 1.20×); on the CPU, where attention over live cells dominates, the same policy is 4.80×; on Phi-3 ($KV/W = 1.8$), 2.64× against a 2.83× ceiling. A baseline proves the law best: KeyDiff at one fixed config is 3.29× on the CPU and 0.27× on the GPU, *slower than the full cache*, its block-wise re-eviction costing more than the smaller cache saves. The lever’s value is a property of the model-backend pair, and the roofline predicts it, yet eviction papers report speedups without stating the KV/W ratio of the measured configuration, the quantity that sets their own ceiling.

- (4) **Compaction feasibility** (§3): the compaction mechanism decides feasibility, not just speed. The state-API round-trip needs a second full context: Phi-3 at 16K is OS-killed on the phone *and* a 24 GB desktop GPU; in-place sliding compaction runs the same configuration inside prefill’s own tensors.

μKV is the design these findings force (§3): attention-scored like SnapKV, but scored *inside* the FlashAttention graph via a side node (prefill +13.7% cold, at vanilla parity sustained and on the GPU, both $n=3$; FA-off prefill costs +33–111%); sequence-level like StreamingLLM, so its budget is realizable and its cache compacts; frozen after prefill; compacted in place. We claim no priority on eviction itself, since KeyDiff [6] also evicts with FlashAttention intact. **Our contributions are:** (1) the first end-to-end measurement of published eviction policies on a phone, each at its own configuration, across throughput, energy, temperature, and cells *actually* freed; (2) three mechanisms we identify and quantify that decide whether eviction works there at all: the realizability gap scaling with model width, the graph split, and the KV/W ceiling. Each is a property of the *engine and the device*, not of any policy, which is why we test each on published baselines as well as on μKV — a mechanism that only bit our own design would be a bug

report, not a finding; (3) μKV, the design those mechanisms force, with its compaction verified byte-identically and its governor characterised as a measured dial; and (4) the measurement traps that make on-device eviction numbers wrong, quantified: a baseline at the wrong budget reads 5.75 rather than 29.07 tok/s, and an unpinned scheduler spreads identical runs by 39% (§4).

2 THREE WAYS THE PHONE BREAKS THE LITERATURE

Realizability. llama.cpp stores KV in one cell array indexed by position; other mobile engines page it in blocks (MLC) or keep per-layer tensors, but in every on-device engine we know of the unit of storage *spans all KV heads*. A per-head policy hands the engine $H \times L$ different keep-sets; the engine can free only the intersection of their complements (Fig. 1a). KV-Compress [7] observes this qualitatively for server tensor layouts; we quantify it on-device, where unanimity gets exponentially rarer as heads multiply. Measured on one build at published budgets: at Llama-1B’s 8 KV heads the per-head class still spreads (Ada-KV retains 35.8%, TOVA 42.0%, SnapKV 91.3%), but at Phi-3’s 32 heads all of them collapse into an 85–100% band (Ada-KV 94.0%, SnapKV 94.5%, H2O 100.0%: 20.4M eviction attempts freeing zero cells), and their throughputs converge to 1.38–1.42×. μKV’s 14/14 reproduces on Phi-3 (15% retained) and Bonsai-8B (16%). Sequence-level selection is immune by construction: across the same width change μKV holds 7.4% to 12.3% and StreamingLLM ~20 to 26%; μKV still evicts on gemma-2’s interleaved-SWA cache (715 cells, 2.4×) where KeyDiff’s eviction never fires at all (0 evicted, 5674 of 5674 retained, 1.02×), but that layout is also the one place in-place compaction declines (§3). A budget is only as real as the cache layout lets it be, and so is a compaction. As models widen, per-head eviction on shared-cell engines trends toward a no-op that costs energy: every Phi-3 per-head cell burns *more* than the full cache. This is invisible in a server

evaluation with per-head paged storage and decisive on a phone: *retained cells*, not nominal budget, set the bandwidth and the memory.

The graph split. The standard way to read attention scores in a graph engine is an eval-callback on the softmax node. On Adreno/Vulkan every FA-off policy's output is corrupted (1–13% degenerate tokens) while vanilla FA-off is clean, vanilla being the one configuration that installs no callback. A three-way control isolates it: no callback 0/978 degenerate, callback reading the tensor 76/609, callback *requesting but never reading* 68/679. The cause is execution granularity: a callback makes the scheduler run the split as separate `ggml_graph_view` fragments, chunked where it is *asked* for a node, before any read; that path is correct on the CPU backend and only wrong on this one. Retrieval confirms it across backends: FA-off policies find the needle on the CPU (H2O 8/14, TOVA 12/14) and collapse on Adreno (3/14, 2/14); FA-on policies are unchanged. The fix is structural: scores must be a node the graph *produces* (μ KV's side node) rather than an interception of one it consumes. Our build now enforces this: on unsafe geometries it auto-promotes prefill-time policies to the side node and refuses per-step ones, recording the verdict in every cell's metadata (Table 2).

The roofline, and why it is a GPU statement only. Per decode token the device reads the weights W plus the live cache. Llama-3.2-1B is heavily GQA-compressed (8 KV heads $\times$ 64 dims = 32 KB/cell): its cache at the 9.7K prompt is 304 MB against 770 MB of weights, so on the *bandwidth-bound GPU* a zero-size cache caps any evictor at 1.40 $\times$, so two policies that both reach $\sim 2k$ cells are indistinguishable by construction, which is exactly what we measure (μ KV 29.05, StreamingLLM 29.07 tok/s). Phi-3 (32 heads $\times$ 96 dims, 12 $\times$ the bytes per cell) has KV/W = 1.83, a 2.83 $\times$ ceiling, and measures 2.64 $\times$ prompt-matched on one build.

On the CPU the same arithmetic does not apply, and the lever inverts. Vanilla decodes at 5.02 tok/s, five times slower than its own weight-bandwidth bound, so the binding resource is not bytes but *attention compute over live cells*. Going 9741 $\rightarrow$ 721 cells is 4.80 $\times$, far above the 1.34 $\times$ a bandwidth model predicts. *The same policy at the same budget is decisive on one processor and nearly irrelevant on the other.*

A second factor, independent of budget: the execution path. Fig. 3 plots what the cache does *not* explain. Across nine policies, deep-throttle residency correlates $r=+0.94$ with whether FlashAttention is off and $r=-0.12$ with cache size; `psi_io` is 0.000 and 6.6–7.2 GB stay free at every point, so no memory-pressure path exists. StreamingLLM and μ KV read an *identical* 823 MB/token (777 vs. 721 cells) and differ 3.6 $\times$ in throughput purely on FA on/off. Normalised by work done, μ KV has the *lowest* throttle exposure of every policy measured (13 s per 1000

tokens against vanilla's 31 and the FA-off cluster's 85–106): it finishes the same 4096 tokens in a fifth of the time, so an equal *fraction* of a shorter run is far less absolute exposure.

3 μ KV

Mechanically, μ KV is four decisions (Fig. 2), each forced by §2: (i) score in-graph (side node, FA never off); (ii) select once at end of prefill, at sequence level. *K itself does not adapt* — it is 1024 in every number in this paper; what adapts is how K is spent. Writing $\bar{a}_p$ for the attention mass at position p averaged over layers, heads and queries, and treating the last $R_{\min}=32$ tokens as the recent region, the α -gate sets $\alpha_a = \sum_{p < N-R_{\min}} \bar{a}_p / \sum_p \bar{a}_p$, clipped to $[0.05, 0.95]$ and floored at 0.70, then splits the budget as $n_{\text{anchor}} = \alpha_a(K - \text{sinks})$ and n_{recent} the remainder. A per-head gate sizes each head's share from its own attention concentration (crediting Ada-KV [4] for the per-head idea), but the emitted keep-set is one set for all heads and layers. The floor is the one tuned constant: on retrieval prompts α_a collapses toward 0.54 and starves the anchors, while on the WikiText prompt it is not binding ($\alpha_a=0.707$ measured); (iii) freeze during decode (only the recent window slides); (iv) compact in place. **μ KV's prefill overhead is a property of the clock, not of the policy.** Protocol, not policy, sets prefill here: the same vanilla configuration spans 172–537 s across our protocols, so cross-campaign prefill comparisons are meaningless and we make none. Cool-gated from cold (Table 1's protocol) μ KV costs +13.7% (vanilla 182 ± 7 s vs. 207 ± 11); warmed and run back-to-back every arm lands in one 240–249 s band. Four attempts to separate the two by matching *temperature* failed, because the cores shed a busy-loop's heat within a second of the load stopping. So we matched the *clock* instead — performance governor pinned at 1632 MHz, verified at the start, middle and end of every run — and the disagreement disappears: vanilla 227.0 ± 1.5 s, μ KV 228.5 ± 2.9 , StreamingLLM 227.4 ± 1.0 ($n=3$, rotated). *At a fixed clock the entire policy effect is +0.7%*, and the 2.7 $\times$ spread we had been reading as a policy difference was governor state. That is what one side node on the last chunk costs; emitting it on every chunk instead cost +106%.

Parameters, and how they were not chosen. The full shipped config is $K=1024$, 4 sinks, observation window 16, SnapKV pool 7, adaptive anchor with $r_{\min}=32$, α -gate floor 0.70. *This one string produced every μ KV number in this paper* — four models, two backends, three benchmarks — and was never re-tuned per model, per task, or per device. Nothing is fitted to an evaluation set: the K sweep that motivated 1024 ran on a different model (Bonsai-8B) and the smallest budget was also the best there, so the choice costs nothing to defend. Compaction is verified at the task level, holding the keep-set identical to the cell (13–21K dropped per run) so compaction

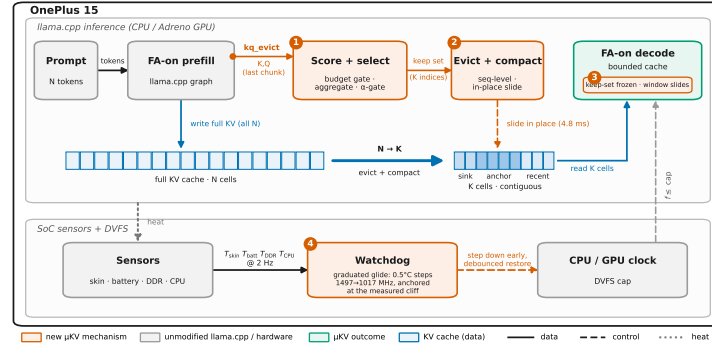


Figure 2: μ KV's pass: in-graph scoring, one keep-set for all heads, in-place compaction. The thermal loop never touches the keep-set.

is the only variable, and comparing generated text byte for byte. **In-place** sliding is output-preserving: 23 of 25 pairs byte-identical across Llama-3.2-1B, Phi-3 and Bonsai-8B, 24 of 25 on the answer span, the two exceptions being a trailing period and post-answer commentary. Perplexity locates the perturbation: across the 23 byte-identical pairs it still moves, by a median of 0.20% and never above 1.69%, so compaction does shift the logits — a compacted cache reduces in a different order — but never past an argmax boundary. That is the mechanism: survivors keep their positions and 2D extents under a slide, so RoPE is unchanged, and a rebuild has no such guarantee. We cannot yet put a number on the **round-trip**: it engages only where in-place declines, which on our models is gemma-2's interleaved-SWA cache alone, and gemma-2's 8192-token window is shorter than these prompts, so both arms degenerate and neither certifies the other. Its cost we can state — a second full context, which is what the OS kills.

4 EVALUATION

Setup and discipline. OnePlus 15; llama.cpp, one build per table; f16 KV (q8_0 is numerically broken on this Adreno driver); Phi-3-mini-128k; every baseline at its own published configuration: SnapKV window 32/pool 7, Ada-KV 2048, TOVA 2048, H2O 20%-of-N resolved per prompt, StreamingLLM start_size 4 + recent_size 2000 = 2004 from its repository defaults. Timed cells: cool-gated (DDR≤35, battery≤33, cores≤45°C; charging off), scheduler-pinned. Pinning matters: unpinned, identical GPU runs spread 28.2–39.2 tok/s (scheduler, not thermal); pinning collapses that to 3%, so any phone-GPU ratio below 1.39× published without it is inside the noise. Energy: USB rail + coulomb counter. Our own earlier comparisons failed this bar and are corrected here. StreamingLLM had run FA-off, uncompacted, at half budget (5.75 vs. faithful 29.07 tok/s); its implementation compacts on every eviction, so compaction is intrinsic to it.

Table 1: Sustained decode, phone CPU. 9.7K prompt + 4K decode, Llama-3.2-1B Q4_K_M, 6 threads. * $n=3$, Latin-square, mean±SD. *Italic:* budget never materialised.

Policy	Config	tok/s	×van	Wall(s)	mWh	ActKV
μ KV*	K=1024	23.91±0.37	4.80	379	595	721
KeyDiff*	K=2048	16.40±0.11	3.29	484	700	2049
Ada-KV	FA on	11.52	2.29	566	845	3489
StreamingLLM*	4+2000	10.25±0.06	2.06	580	781	2001
SnapKV	FA off	6.83	1.36	849	1180	5931
Ada-KV	FA off	6.79	1.35	856	1194	3485
KeyDiff	K=8192	6.43	1.28	812	1261	8195
H2O	FA off	6.45	1.29	856	1262	<i>9151</i>
TOVA	FA off	5.92	1.18	917	1200	4093
SnapKV	FA on	5.38	1.07	953	1346	8893
vanilla*	full cache	4.99±0.01	1.00	1004	1431	9741
<i>μKV on the other CPU models, same protocol, each vs. its own vanilla:</i>						
Phi-3-mini	K=1024	5.53	4.54	841	928	929
Bonsai-8B (1-bit)	K=1024	4.46	4.43	1996	2415	789
vs. KeyDiff	K=2048	2.01	2.00	—	—	2048
gemma-2-2b (iSWA)	K=1024	9.28	2.40	—	—	715

Table 2: Phone GPU. 12K prompt + 4K decode; Llama rows $n=3$. Every generation is text-graded. On Phi-3 the gate promotes SnapKV and Ada-KV and refuses H2O and TOVA (\$2).

Policy	tok/s	vs van.	mWh	Cells
vanilla	24.28±0.12	1.00×	437	9741
μ KV	29.05±0.13	1.20×	400	723
StreamingLLM	29.07±0.18	1.20×	401	2005
per-head (4) / KeyDiff	2.9–4.8 / 1.6	0.11–0.19×	0.06×	— 2049–6592
<i>Phi-3-mini block (KV/W= 1.83); compare within a block only:</i>				
vanilla	5.05	1.00×	1936	11157
μ KV (in-place)	13.36	2.64×	1186	878
StreamingLLM	10.60	2.10×	1126	2000
Ada-KV (promoted)	9.13	1.81×	1430	4326
SnapKV (promoted)	5.80	1.15×	1837	10862
KeyDiff (2K)	1.36	0.27×	2402	2048
H2O / TOVA	refused by gate: FA-off capture corrupt at head_dim 96			

Speed and energy. Table 1. μ KV reaches $4.80 \pm 0.07 \times$ vanilla on 721 live cells at 595 mWh (−58%); the best baseline, KeyDiff at its smallest published budget, is $1.46 \times$ slower on $2.8 \times$ the cells and 18% more energy. The margin survives sustained use: six back-to-back 16K generations with no cooling hold μ KV at 23.0–23.3 tok/s (\$5), so the speedup is not a cold-start artefact. Two baselines were

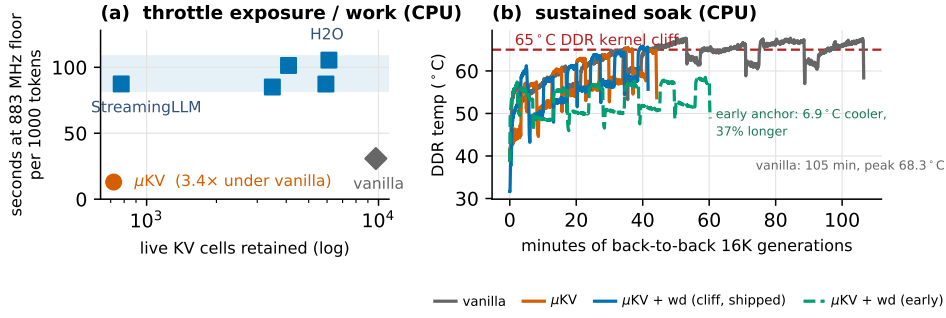


Figure 3: (a) Throttle exposure per 1000 decoded tokens, seconds at the 883 MHz floor. (b) Six-generation soak, one cool gate then none.

Table 3: Quality at each policy’s own published budget (LongBench token-F1, $n=15$). Retention sits beside every score; *italic* marks a budget that never materialised.

Policy	qasper		hotpotqa	
	F1	retained	F1	retained
vanilla (full cache)	20.29	100%	36.00	100%
μ KV (K=1024)	17.32	18.2%	29.79	13.7%
KeyDiff 8K [§]	—	—	36.39	62.3%
KeyDiff 6K [§]	—	—	29.33	48.4%
KeyDiff 2K	23.46	47.6%	15.84	20.0%
StreamingLLM [†]	19.31	46.5%	23.57	19.5%
SnapKV	20.05	97.0%	40.00	89.1%
Ada-KV	20.05	85.7%	40.00	53.7%
H2O	17.84	91.5%	39.79	91.8%
TOVA	20.05	91.2%	39.89	57.4%

Table 4: Retrieval, phone GPU. 14 stimuli $\times$ 7 depths, retention beside every score. *Italic*: budget never materialised. [‡]FA-off, invalid on this backend (§2).

Policy	Budget	NIAH	Retained 4K / 8K
vanilla	—	14/14	100% 100%
μ KV	1024	14/14	25.2% 13.0%
KeyDiff	2048	14/14	65.6% 33.6%
KeyDiff [§]	6144	14/14	100% 100%
Ada-KV	1024	12/14	76.9% 52.2%
SnapKV	1024	12/14	100% 98.2%
StreamingLLM [†]	2004	7/14	64.2% 32.9%
H2O [‡]	1024	3/14	100% 99.3%
TOVA [‡]	1024	2/14	85.2% 60.7%

previously handicapped by our harness and are corrected here: Ada-KV scored from the in-graph side node with its own window runs 1.70 $\times$ faster than through the FA-off capture (11.52 vs. 6.79) at identical retention, while SnapKV runs *slower* FA-on (5.38 vs. 6.83) and retains more (8893 vs. 5931 cells), its windowed scores yielding a larger per-head union. We report both configurations rather than the flattering one.

Quality, read with retention (Table 3). Per-head policies appear to win (SnapKV 40.00 on hotpotqa vs. μ KV’s 29.79) while retaining 89.1% of the cache; six of their fifteen generations are *byte-identical to vanilla*’s: the realizability gap reappearing on the quality axis.

KeyDiff at its own published budgets splits. Its 8K claim reproduces: 36.39 F1 vs. a 36.00 full cache (+1.1%; claim $\leq 0.04\%$), retaining 62.3%. Its 6K claim does not survive our prompt lengths: 29.33 F1, -18.5% vs. a claimed $\leq 1.5\%$, because hotpotqa’s median prompt is 15.5K tokens, so 6144 cells retain only 48%, far harsher than the LongBench average behind their number. The comparison that matters: μ KV reaches **29.79 F1 on 1763 live cells where KeyDiff-6K reaches 29.33 on 5886**, equal quality on 3.3 $\times$ less cache; KeyDiff needs 62% retention to match full where μ KV delivers 83% of it at 13.7%.

The pattern is not a phone artefact. Repeating hotpotqa on an RTX 4500 ($n=50$) reproduces it: per-head policies retain 33–81% there too (shared cell array on every backend; vanilla 42.14 F1, SnapKV 41.12 at 80.8%). At 8.7% retention μ KV scores 40.72, within 1.4F1 of full, while StreamingLLM, the only other policy in that retention class, loses 8.8 F1. On qasper six policies *beat* the full cache (distractor filtering, §5), so qasper cannot rank eviction quality and we do not use it to.

Why the cache at all: the privileged-lever boundary. Three actuators cut decode energy here, measured under one protocol, and the ranking that matters is *reachability*, not effect size. Capping the GPU clock is the strongest thermal lever, since 1200 $\rightarrow$ 902 MHz costs no throughput (30.0 vs. 30.3 tok/s: decode is bandwidth-bound, so the clock is idle headroom) and drops peak DDR 16.6°C, but it is a write to `/sys/class/kgsl/kgsl-3d0/max_gpuclk`; scheduler pinning removes a 39% dispatch bimodality but needs nice -20 . **Both require root; an application can reach neither.** KV-cache size is the only actuator inside the inference engine. It is the weakest of the three, the only one that costs quality, and the only one that makes the device *hotter* (k-pct 10 runs 5.8°C above k-pct 20). It

Table 5: Governor, phone CPU soak. Six generations, one cool gate then none. *Anchor* is the battery temperature at which the ladder’s first step fires.

Run	Anchor (°C)	Gen 1 (tok/s)	Steady (tok/s)	Floor (%)	Peak DDR
vanilla	—	4.97	4.99	3.3	68.3
vanilla	47	5.01	4.98	3.6	67.2
μ KV	—	23.32	23.14	8.2	65.6
μ KV	47	26.95	22.97	6.4	66.0
μ KV	42	27.12	21.07	5.4	63.3
μ KV	35	18.00	15.95	2.4	58.7

is also the only one a runtime may use, which is why a battery-aware cache policy is worth building, and why its tiers must come from a measured frontier rather than chosen thresholds.

The frontier, and what it refuted. A controller reads state-of-charge and sets K as a fraction of prompt length ($K=1947/974/487$, $n=3$ rotated). Quality per tier ($n=15$): gasper 17.5/10.9/10.6 vs. 20.3 full; hotpotqa flat at 29.8 vs. 36.0, so the price of a tier step is task-dependent, zero on factoid QA, -6.7 F1 on multi-fact QA. The 5% rung is dominated and deleted.

But the lever itself does not survive its own test. We re-ran the ladder cooled, scheduler-pinned and interleaved ($n=3$) with the GPU clock held at 902 MHz, so that cache budget is the *only* variable. Halving the cache ($K=1947 \rightarrow 974$, $1379 \rightarrow 688$ live cells) changes decode energy by $+1.0\%$, inside the 8–10% run-to-run spread of both arms, while buying $+13.1\%$ throughput, costing $+3.5^\circ\text{C}$, and costing 6.7 F1. *On the phone GPU the cache tier is not an energy actuator at all.* A battery-aware controller that shrinks the cache gives up quality and runs hotter for no energy return.

The regime decides, and the controller reads the wrong variable. On the CPU, where decode is attention-bound over live cells, $9741 \rightarrow 721$ cells is -61% energy (601 vs. 1544 mWh); on the GPU, where 800 MB of weights dominate 45 MB of cache, the same reduction is worth nothing. KV/W tells you which regime you are in, and our controller, as built, *adapts on the GPU* (1379/688/342 cells) and *declines on the CPU* (341 cells at every battery level), which is exactly backwards. The corrected design conditions on KV/W, not on the battery alone. We report this as a negative result rather than repair it silently: it is the clearest evidence in this paper that a lever’s value is a property of the model–backend pair and not of the policy.

5 DISCUSSION: NEGATIVES AND OPEN PROBLEMS

HotMobile asks for what did *not* work; each item below is a measurement. **Cache is not a thermal actuator:** at

identical GPU clock the busier (evicting) run is 8°C hotter; frequency remains the thermal lever. **The watchdog buys clock while the device is cool, not while it is hot** (Table 5). The vendor gives a cliff, not a glide: a flat 1498 MHz from battery $46\text{--}49^\circ\text{C}$, then 883 the instant it hits 50.0 (measured). Our governor replaces it with a staircase (0.5°C steps, $1497 \rightarrow 1017$ MHz, immediate-up/debounced-down) anchored there. The gain is *front-loaded* and *conditional on the workload*: μ KV’s cool generation gains $+15.5\%$, because at tier 0 the governor writes `cpuinfo_max_freq`, releasing a ceiling the vendor HAL had already lowered — it is reduce-only against the *hardware* maximum, not the vendor’s current setting. The same governor on vanilla gains $+0.9\%$: that run is bandwidth-bound and cannot spend released clock, so the headroom returns as temperature (-2.4°C mean DDR). *The governor converts thermal headroom into whichever resource the workload lacks*, which is why it needs eviction to appear as speed. Both μ KV arms then converge to ~ 1378 MHz, so the watchdog is free rather than a sustained win. The anchor is a monotone dial whose sign depends on the backend: earlier trades throughput for temperature on the CPU (Table 5), while on the GPU its ends trade different currencies: the early anchor (36°C) gains $+10.8\%$ (residency $41.0 \rightarrow 37.0\%$), the late one (47°C) buys residency $41.0 \rightarrow 28.5\%$ for $+0.8\%$. A capped clock is idle headroom on a bandwidth-bound backend and paid work on a compute-bound one. **Peak RSS rises under eviction** (the engine allocates at full context up front): the memory win is retained cells and bandwidth, never footprint. **Perplexity cannot referee eviction:** on a disjoint slice every evictor we can score *beats* the full cache (μ KV 22.7 vs. 23.5; 7.9 vs. 8.1 on Phi-3): PPL certifies harmlessness; only retrieval/QA separate policies.

Future work, for this workshop to argue about.

(1) *Scores as graph outputs*: the split of $\$2$ makes callback scoring unsafe, and per-head paging (shipping server-side) is not on phones yet; the head-count law quantifies what it must fix. (2) Phase-aware DVFS exists for servers; no *shipped* mobile governor knows the runtime’s phase, a gap worth a measured 16.6°C of free headroom. (3) Offloading (KVSwap, DynaKV) and eviction should be *composed*: offloaders write the whole cache and select at recall, where an evictor’s scores can triage at *write* time (keep/park/discard).

REFERENCES

- [1] Z. Zhang et al. H2O. *NeurIPS*, 2023.
- [2] M. Oren et al. TOVA: Transformers are multi-state RNNs. *EMNLP*, 2024.
- [3] Y. Li et al. SnapKV. *NeurIPS*, 2024.
- [4] Y. Feng et al. Ada-KV. *NeurIPS*, 2025.
- [5] G. Xiao et al. Efficient streaming LMs with attention sinks. *ICLR*, 2024.
- [6] J. Park et al. KeyDiff: Key similarity-based KV cache eviction. *NeurIPS*, 2025.
- [7] I. Rehag. KV-Compress. arXiv:2410.00161, 2024.
- [8] KVSwap: Disk-aware KV cache offloading on-device. *MobiSys*, 2026.
- [9] DynaKV: Smartphone KV offloading to UFS. 2025.
- [10] MobiLoRA: Context-aware KV cache optimization for mobile LoRA inference. *ACL*, 2025.
- [11] G. Gerganov et al. llama.cpp (github.com/ggml-org).